Module 22

Relational Database Design/2

Milav Dabgar

www.milav.in

March 17, 2026

Module Recap

- Identified the features of good relational design

Module Recap

- Identified the features of good relational design
- Familiarized with the First Normal Form

Module Objectives

Goals

- To Introduce Functional Dependencies

Module Outline

- Functional Dependencies

Functional Dependencies

Goal: Devise a Theory for Good Relations

- Decide whether a particular relation R is in 'good' form.

Goal: Devise a Theory for Good Relations

- Decide whether a particular relation R is in 'good' form.
- In the case that a relation R is not in 'good' form, decompose it into a set of relations $\{R_1, R_2, \ldots, R_n\}$ such that

Goal: Devise a Theory for Good Relations

- Decide whether a particular relation R is in 'good' form.
- In the case that a relation R is not in 'good' form, decompose it into a set of relations $\{R_1, R_2, \ldots, R_n\}$ such that
  - each relation is in good form

Goal: Devise a Theory for Good Relations

- Decide whether a particular relation R is in 'good' form.
- In the case that a relation R is not in 'good' form, decompose it into a set of relations $\{R_1, R_2, \ldots, R_n\}$ such that
  - each relation is in good form
  - the decomposition is a lossless-join decomposition

Goal: Devise a Theory for Good Relations

- Decide whether a particular relation R is in 'good' form.
- In the case that a relation R is not in 'good' form, decompose it into a set of relations $\{R_1, R_2, \ldots, R_n\}$ such that
  - each relation is in good form
  - the decomposition is a lossless-join decomposition
- The theory is based on:

Goal: Devise a Theory for Good Relations

- Decide whether a particular relation R is in 'good' form.
- In the case that a relation R is not in 'good' form, decompose it into a set of relations $\{R_1, R_2, \ldots, R_n\}$ such that
  - each relation is in good form
  - the decomposition is a lossless-join decomposition
- The theory is based on:
  - Functional dependencies

Goal: Devise a Theory for Good Relations

- Decide whether a particular relation R is in 'good' form.
- In the case that a relation R is not in 'good' form, decompose it into a set of relations $\{R_1, R_2, \ldots, R_n\}$ such that
  - each relation is in good form
  - the decomposition is a lossless-join decomposition
- The theory is based on:
  - Functional dependencies
  - Multivalued dependencies

Goal: Devise a Theory for Good Relations

- Decide whether a particular relation R is in 'good' form.
- In the case that a relation R is not in 'good' form, decompose it into a set of relations $\{R_1, R_2, \ldots, R_n\}$ such that
  - each relation is in good form
  - the decomposition is a lossless-join decomposition
- The theory is based on:
  - Functional dependencies
  - Multivalued dependencies
  - Other dependencies

Functional Dependencies

Definition

- Constraints on the set of legal relations

Functional Dependencies

Definition

- Constraints on the set of legal relations
- Require that the value for a certain set of attributes determines uniquely the value for another set of attributes

Functional Dependencies

Definition

- Constraints on the set of legal relations
- Require that the value for a certain set of attributes determines uniquely the value for another set of attributes
- A functional dependency is a generalization of the notion of a key

Functional Dependencies (2)

- Let R be a relation schema $\alpha \subseteq R$ and $\beta \subseteq R$

Functional Dependencies (2)

- Let R be a relation schema $\alpha \subseteq R$ and $\beta \subseteq R$
- The functional dependency or FD

$$\alpha \rightarrow \beta$$

holds on R if and only if for any legal relations $r(R)$, whenever any two tuples t_1 and t_2 of r agree on the attributes α, they also agree on the attributes β. That is,

$$t_1[\alpha] = t_2[\alpha] \Rightarrow t_1[\beta] = t_2[\beta]$$

Functional Dependencies (2)

- Let R be a relation schema $\alpha \subseteq R$ and $\beta \subseteq R$
- The functional dependency or FD

$$\alpha \rightarrow \beta$$

holds on R if and only if for any legal relations $r(R)$, whenever any two tuples t_1 and t_2 of r agree on the attributes α, they also agree on the attributes β. That is,

$$t_1[\alpha] = t_2[\alpha] \Rightarrow t_1[\beta] = t_2[\beta]$$

- Example: Consider $r(A, B)$ with the following instance of r.

A	B
1	4
1	5
3	7

Functional Dependencies (2)

- Let R be a relation schema $\alpha \subseteq R$ and $\beta \subseteq R$
- The functional dependency or FD

$$\alpha \rightarrow \beta$$

holds on R if and only if for any legal relations $r(R)$, whenever any two tuples t_1 and t_2 of r agree on the attributes α, they also agree on the attributes β. That is,

$$t_1[\alpha] = t_2[\alpha] \Rightarrow t_1[\beta] = t_2[\beta]$$

- Example: Consider $r(A, B)$ with the following instance of r.

A	B
1	4
1	5
3	7

Functional Dependencies (3)

- K is a superkey for relation schema R if and only if $K \rightarrow R$

Functional Dependencies (3)

- K is a superkey for relation schema R if and only if $K \rightarrow R$
- K is a candidate key for R if and only if

Functional Dependencies (3)

- K is a superkey for relation schema R if and only if $K \rightarrow R$
- K is a candidate key for R if and only if
  - $K \rightarrow R$ and

Functional Dependencies (3)

- K is a superkey for relation schema R if and only if $K \rightarrow R$
- K is a candidate key for R if and only if
  - $K \rightarrow R$ and
  - for no $\alpha \subset K$, $\alpha \rightarrow R$

Functional Dependencies (3)

- K is a superkey for relation schema R if and only if $K \rightarrow R$
- K is a candidate key for R if and only if
  - $K \rightarrow R$ and
  - for no $\alpha \subset K$, $\alpha \rightarrow R$
- Functional dependencies allow us to express constraints that cannot be expressed using superkeys. Consider the schema:

```
inst_dept(ID, name, salary, dept_name, building, budget)
```

Functional Dependencies (3)

- K is a superkey for relation schema R if and only if $K \rightarrow R$
- K is a candidate key for R if and only if
  - $K \rightarrow R$ and
  - for no $\alpha \subset K$, $\alpha \rightarrow R$
- Functional dependencies allow us to express constraints that cannot be expressed using superkeys. Consider the schema:

  `inst_dept(ID, name, salary, dept_name, building, budget)`
- We expect these functional dependencies to hold:

$$\texttt{dept_name} \rightarrow \texttt{building}$$
$$\texttt{dept_name} \rightarrow \texttt{budget}$$
$$\texttt{ID} \rightarrow \texttt{budget}$$

Functional Dependencies (3)

- K is a superkey for relation schema R if and only if $K \rightarrow R$
- K is a candidate key for R if and only if
  - $K \rightarrow R$ and
  - for no $\alpha \subset K$, $\alpha \rightarrow R$
- Functional dependencies allow us to express constraints that cannot be expressed using superkeys. Consider the schema:

  `inst_dept(ID, name, salary, dept_name, building, budget)`
- We expect these functional dependencies to hold:

$$\texttt{dept_name} \rightarrow \texttt{building}$$
$$\texttt{dept_name} \rightarrow \texttt{budget}$$
$$\texttt{ID} \rightarrow \texttt{budget}$$

- but would not expect the following to hold:

$$\texttt{dept_name} \rightarrow \texttt{salary}$$

Functional Dependencies (4)

- We use functional dependencies to:

Functional Dependencies (4)

- We use functional dependencies to:
  - test relations to see if they are legal under a given set of functional dependencies.

Functional Dependencies (4)

- We use functional dependencies to:
  - test relations to see if they are legal under a given set of functional dependencies.
  - If a relation r is legal under a set F of functional dependencies, we say that r satisfies F

Functional Dependencies (4)

- We use functional dependencies to:
  - test relations to see if they are legal under a given set of functional dependencies.
  - If a relation r is legal under a set F of functional dependencies, we say that r satisfies F
  - specify constraints on the set of legal relations

Functional Dependencies (4)

- We use functional dependencies to:
  - test relations to see if they are legal under a given set of functional dependencies.
  - If a relation r is legal under a set F of functional dependencies, we say that r satisfies F
  - specify constraints on the set of legal relations
  - We say that F holds on R if all legal relations on R satisfy the set of functional dependencies F

Functional Dependencies (4)

- We use functional dependencies to:
  - test relations to see if they are legal under a given set of functional dependencies.
  - If a relation r is legal under a set F of functional dependencies, we say that r satisfies F
  - specify constraints on the set of legal relations
  - We say that F holds on R if all legal relations on R satisfy the set of functional dependencies F
- **Note**: A specific instance of a relation schema may satisfy a functional dependency even if the functional dependency does not hold on all legal instances

Functional Dependencies (4)

- We use functional dependencies to:
  - test relations to see if they are legal under a given set of functional dependencies.
  - If a relation r is legal under a set F of functional dependencies, we say that r satisfies F
  - specify constraints on the set of legal relations
  - We say that F holds on R if all legal relations on R satisfy the set of functional dependencies F
- **Note**: A specific instance of a relation schema may satisfy a functional dependency even if the functional dependency does not hold on all legal instances
  - For example, a specific instance of `instructor` may, by chance, satisfy `name` $\rightarrow$ `ID`

Functional Dependencies (4)

- We use functional dependencies to:
  - test relations to see if they are legal under a given set of functional dependencies.
  - If a relation r is legal under a set F of functional dependencies, we say that r satisfies F
  - specify constraints on the set of legal relations
  - We say that F holds on R if all legal relations on R satisfy the set of functional dependencies F
- **Note**: A specific instance of a relation schema may satisfy a functional dependency even if the functional dependency does not hold on all legal instances
  - For example, a specific instance of `instructor` may, by chance, satisfy `name` $\rightarrow$ `ID`
  - In such cases we do not say that F holds on R

Functional Dependencies (5)

- A functional dependency is trivial if it is satisfied by all instances of a relation

Functional Dependencies (5)

- A functional dependency is trivial if it is satisfied by all instances of a relation
- Example:

Functional Dependencies (5)

- A functional dependency is trivial if it is satisfied by all instances of a relation
- Example:
  - `ID, name` $\rightarrow$ `ID`

Functional Dependencies (5)

- A functional dependency is trivial if it is satisfied by all instances of a relation
- Example:
  - `ID, name` $\rightarrow$ `ID`
  - `name` $\rightarrow$ `name`

Functional Dependencies (5)

- A functional dependency is trivial if it is satisfied by all instances of a relation
- Example:
  - `ID, name` $\rightarrow$ `ID`
  - `name` $\rightarrow$ `name`
- In general, $\alpha \rightarrow \beta$ is trivial if $\beta \subseteq \alpha$.

Functional Dependencies (6)

- Functional dependencies are:

StudentID	Semester	Lecture	TA
1234		Numerical Methods	John
1221		Numerical Methods	Smith
1234		Visual Computing	Bob
1201		Numerical Methods	Peter
1201		Physics II	Simon

Functional Dependencies (6)

- Functional dependencies are:
  - `StudentID` $\rightarrow$ `Semester`

StudentID	Semester	Lecture	TA
1234		Numerical Methods	John
1221		Numerical Methods	Smith
1234		Visual Computing	Bob
1201		Numerical Methods	Peter
1201		Physics II	Simon

Functional Dependencies (6)

- Functional dependencies are:
  - `StudentID` $\rightarrow$ `Semester`
  - `StudentID, Lecture` $\rightarrow$ `TA`

StudentID	Semester	Lecture	TA
1234		Numerical Methods	John
1221		Numerical Methods	Smith
1234		Visual Computing	Bob
1201		Numerical Methods	Peter
1201		Physics II	Simon

Functional Dependencies (6)

- Functional dependencies are:
  - `StudentID` $\rightarrow$ `Semester`
  - `StudentID, Lecture` $\rightarrow$ `TA`
  - {`StudentID, Lecture`} $\rightarrow$ {`TA, Semester`}

StudentID	Semester	Lecture	TA
1234		Numerical Methods	John
1221		Numerical Methods	Smith
1234		Visual Computing	Bob
1201		Numerical Methods	Peter
1201		Physics II	Simon

Functional Dependencies (7)

- Functional dependencies are:

Employee ID	Employee Name	Department ID	Department Name
0001	John Doe		Human Resources
0002	Jane Doe		Marketing
0003	John Smith		Human Resources
0004	Jane Goodall		Sales

Functional Dependencies (7)

- Functional dependencies are:
  - `EmployeeID` $\rightarrow$ `EmployeeName`

Employee ID	Employee Name	Department ID	Department Name
0001	John Doe		Human Resources
0002	Jane Doe		Marketing
0003	John Smith		Human Resources
0004	Jane Goodall		Sales

Functional Dependencies (7)

- Functional dependencies are:
  - `EmployeeID` $\rightarrow$ `EmployeeName`
  - `EmployeeID` $\rightarrow$ `DepartmentID`

Employee ID	Employee Name	Department ID	Department Name
0001	John Doe		Human Resources
0002	Jane Doe		Marketing
0003	John Smith		Human Resources
0004	Jane Goodall		Sales

Functional Dependencies (7)

- Functional dependencies are:
  - `EmployeeID` $\rightarrow$ `EmployeeName`
  - `EmployeeID` $\rightarrow$ `DepartmentID`
  - `DepartmentID` $\rightarrow$ `DepartmentName`

Employee ID	Employee Name	Department ID	Department Name
0001	John Doe		Human Resources
0002	Jane Doe		Marketing
0003	John Smith		Human Resources
0004	Jane Goodall		Sales

Functional Dependencies (9): Closure of a Set of FDs

- $F = \{A \rightarrow B, B \rightarrow C\}$

Functional Dependencies (9): Closure of a Set of FDs

- $F = \{A \rightarrow B, B \rightarrow C\}$
- $F^+ = \{A \rightarrow B, B \rightarrow C, A \rightarrow C\}$

Module Summary

Summary

- Introduced the notion of Functional Dependencies

Slides used in this presentation are borrowed from `http://db-book.com/` *with kind permission of the authors.*
Edited and new slides are marked with 'PPD'.